Java

JVM
↳ execution
byte code

1. highlevel programming language.

J2EE
↳ edition
enteropraise

J2ME
↳ Mobile Application

Java is a object oriented programme.

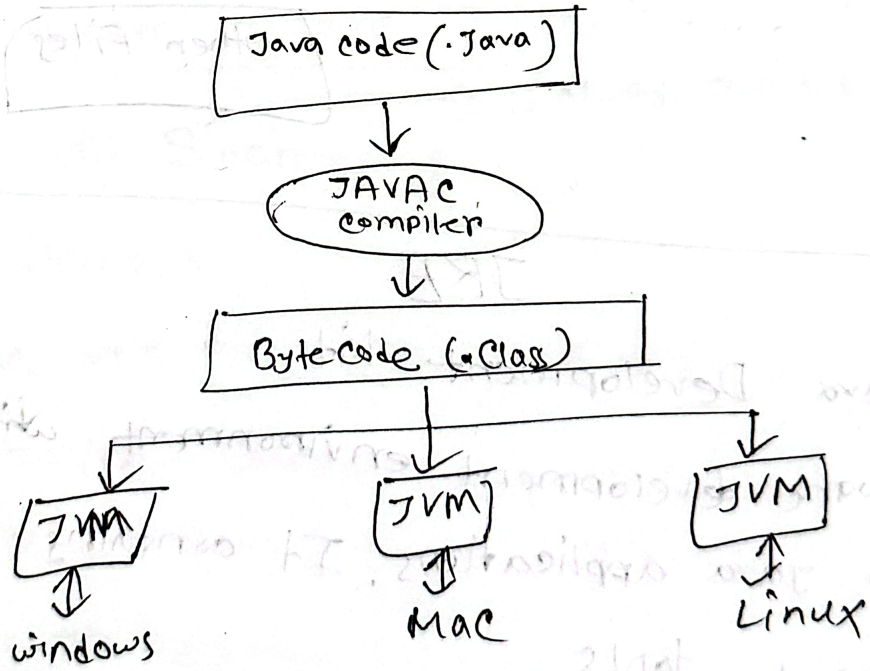
easy for web/internet

Robust
↳ error finding easy

Java platform independent.

1. Why Java is platform independent?

Main.Java
Min. class

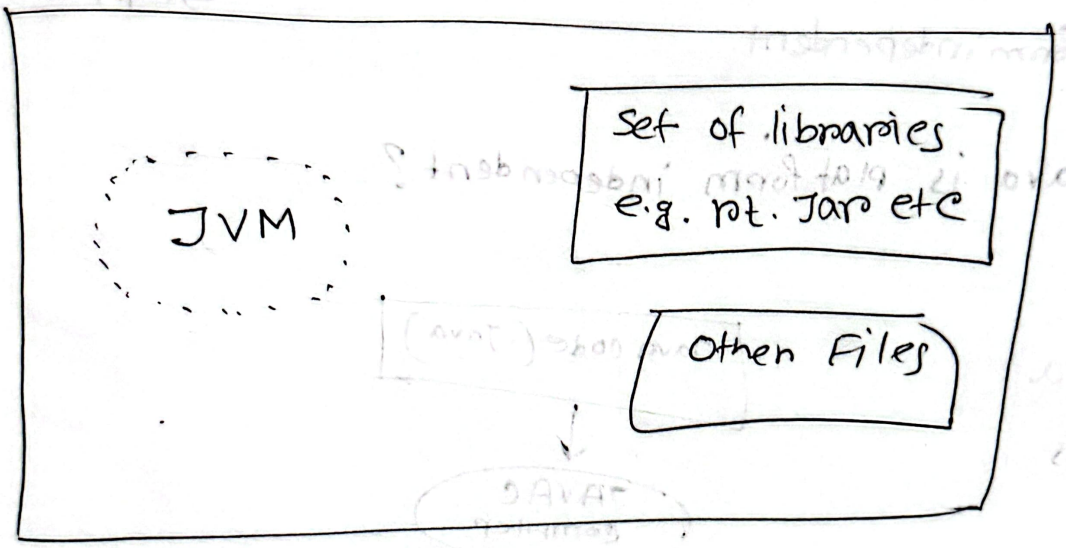


JRE ⇒ Java Runtime environment

Gis a set of software tools which are used for developing Java applications.

it is the implementation of JVM. It physical exists.

It contains a set of libraries.



JRE

JDK ⇒ Java Development kit

Gis a software development environment which is used to develop Java applications. It contains JRE + development tools.

whole complete packeg is JDK

Program structure.

A.Java

```
Public class A {
    public static void main (String[] args) {
        System.out.println ("Hello World");
    } //end of main
}
```

ln is for new line

always System. S capital 20.

println function

Output: Hello World

Second program in Java

```
import Java.util.Scanner;
```

S always capital 20.

```
pubic class Demo {
    public static void main (string[] args) {
        Scanner b = New Scanner (system.in);
        System.out.println ("Enter any number:");
        int num = b.nextInt ();
        System.out.println ("The number entered by user :" + num);
    }
}
```

input लेना है

Classname object = new classname ( )

function
Method লেখার শুরুতে
বড় হাতের লিখতে হবে।

Object. class in Java

⇒

```
Class Student {
    String name;
    int ID;
Void display ( )
    {
System.out.println ("Student Name: " +name);
    }
}
Public Static void main (String[] args) {
  Student S1 = new Student ( ); //object creation in main function
  S1. Name = "Sohan";
  S1. ID = 20255103246;
  S1. display ( );
}
```

Modifiers in Java.

tpointtech.

access modifiers:

Public, protected, private, default.

- Public ↙ → all can access
- protected ↳ → Child class can access
- default ↳ → same package থেকে access করতে পারবে, other same package এর বাইরে access করতে পারবে না।

Non access modifiers:

final, abstract, strict.

5. print (1-100) even numbers sumation.

Sum.Java

```
public class sum {
public static void main (String[] args)
int s=0;
  for(int i=1; i<=100; i++) {
  if (i%2 ==0) {
    s += i;
System.out.println ("sum of even numbers : "+s);
  }
}
```

mPhy

```
class Dog{
    String name;
    void bark(){
        System.out.println(name + "says Woof!")
    }
}
public class main{
    public static void main(String[] args){
        Dog myDog = new Dog();
        myDog.name = "Rocky";
        MyDog.bark();
    }
}
```

```
Class Teacher {
    String name;
    String Dept;
  void display ()
    {
  System. out. println ("Teacher Name: "+name);
  System. out. println ("Teacher Dept : "+Dept);
    }
}
public class main {
public static void main (String [] args) {
    Teacher T1 = new Teacher();
      T1. Name = "NAFIZ";
      T1. Dept  = "EEE";
      T1. display ();
Teacher T2 = new Teacher ();
  T2. Name = "Farook_Hasan";
  T2. Dept = "civil"
  T2. display ();
```

lab -1

input Function

- Character নেয়ার জন্য

```
Chara ch = b.next().charAt(0);
```

String নেয়ার জন্য

```
next()
next line()
```

array

```
int[] arr = {1, 2, 3, 4};
for (int i = 0; i<arr.length; i++)
    System.out.println(arr[i]);
```

take an array, ▷ build a loop even numbers sum.

for array size declaration:

```
int[] arr = new int[5];
```

14-09-26

while

```
int a = 10 ; b = 20 ;
while (a>b) {
System.out.println ("This not enough") ;
}
```

Do while

```
class Dowhile {
    public static void main
```

for

Initializing an object in Java

1. By Referance variable
2. By Method
3. By constructor → object ক্ষে value initilize করার জন্য constructor use করা হয়।

class Employee

- employee ID
- Name
- Designation
- Monthly. salary
- constructor()
- calculate annual Salary()
- display()

creat Two object of employee class.

```
Public Class ~~student~~ Employee {
:Private int employeeID;
Private String name;

Class Employee {
int employeeID;
String name;
String Designation;
int Monthly_salary;
Employee (int employeeID, String name, String
          Designation, int Monthly_salary) {
this. employeeID = employeeID;
this. name = name;
this. Designation = Designation;
this. Monthly_salary = Monthly_salary;
}
int calculateAnnual salary. ( ) {
return Monthly_salary * 12;
}
```

20/20

```
void Display (){
System.out.println ("Employee ID : " + employeeID);
System.out.println ("Name : " + name);
System.out.println ("Designation : " + Designation);
System.out.println ("Monthly Salary :" + Monthly_Salary);
System.out.println("Annual Salary :" + calculateAnnualSalary ());
}

public class main {
    public static void main(String[] args){
        Employee Empl = new Employee (101, "Sohan",
                "Developer", 25000);
        Employee Emp2 = new Employee (203, "Resalat",
                "HR", 30000);
        Emp1.Display ();
        Emp2.Display ();
    }
}
```

Constructor creat না করলেও object creat করা difficult

constructor auto call হয়।

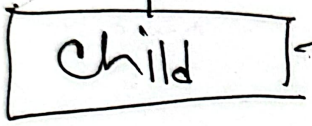
This keyword

```
class A {
A() {
this (5);
system.out.println ("hello a");
}
A(int x) {
System.out.println (x);
}
}
class main
pubic static void main (string[] args[] ) {
    A a = new A ();
}
}
```

Inheritance

→ another names: Base class, Super class

inherits



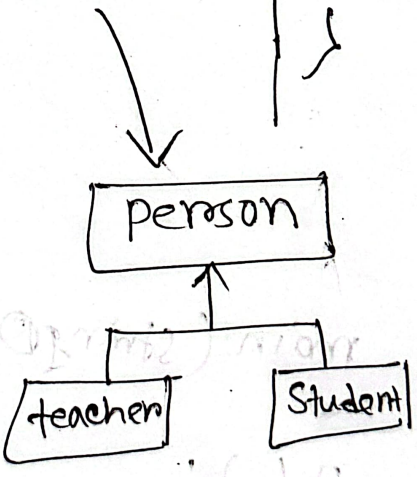
→ another name : Subclass / Derived class

child extends parent

class subclass-name
extends Superclass-name
{
// Methods and files.
}

IS-A ⇒ Ex.=Programmer is a employee

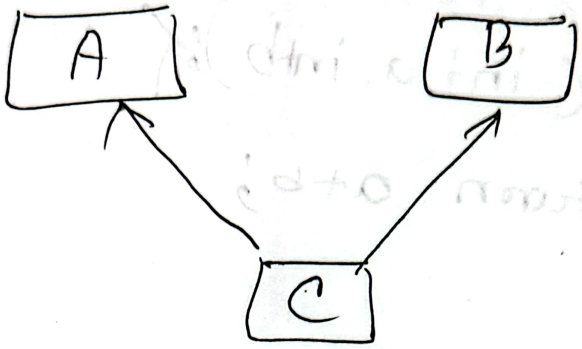
HAS-A



Inheritance:

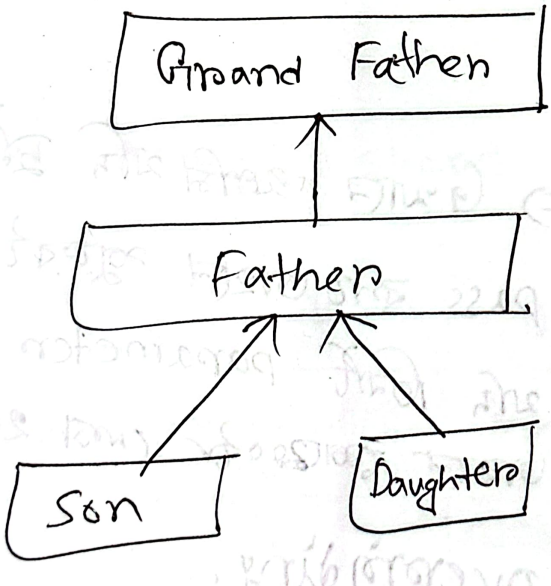
- Single = only one parent and one child
- multilevel = Parent, child, grandchild
- Hierarchical = Inheritance, one parent, multiple child
- Hybrid = Multiple inheritance.

Multipler inheritance:



This is not supportive in JAVA

~~by~~ Hybrid Inheritance



Method overloading

```
Class A {
    int sum(int a, int b){
        return a+b;
    }
    int sum (int a, int b, int c){
        return a+b+c;
    }
}
```

ob.sum( ) ⇒ এখানে যদি দুটি parameter pass করে তাহলে প্রথমটা চলবে এবং যদি তিনটি parameter pass করে তাহলে দ্বিতীয়টি চলবে।

Method overriding.

Inheritance থাকলেই Method overriding করা possible

⇒
```
class A
{
    int sum (int a, int b){
        return a+b;
    }
```

```
}
class B extends A {
    int sum (int a, int b) {
        return a-b;
    }
}
```

Ob.B ( ) [যদি বিয়োগ করতে হয় তবে]

যদি যোগ করতেহয় Ob.A ( );

I have a class Shape; Method get area.

Overloading	Overriding
① inheritance লাগেনা	inheritance লাগে
	~~Implementation~~

```
Class Shape {
    public double getArea() {
        return 0.0;
    }
}

Class Rectangle extends Shape {
    private double length;
    private double width;

public Rectangle(double length, double width) {
(constructor)
        This.length = length;
        this.width = width;
    }
    public double getArea() {
        return length * width;
    }
}

public ~~main~~ class Main {
  public static void main (String[] args) {
  Rectangle rec = new Rectangle (5.6, 3.0);
```

(a)

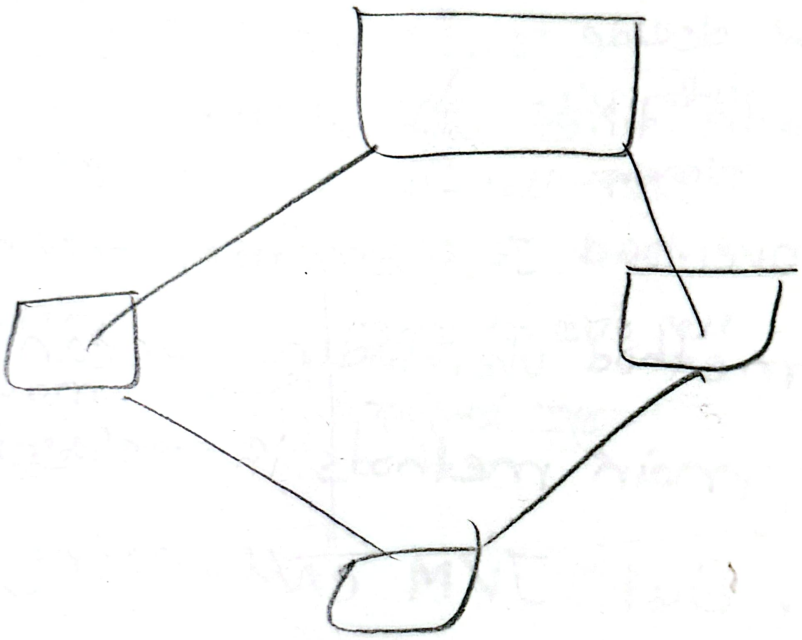
System.out.println("Area of Rectangle: "+rec.getArea());

}

}

~~Inheritance:~~

Diamond Problem



Static key word: যদি কোন member static হিসেবে define ~~decide~~ করা থাকে তাহলে main function থেকে ডাইরেক্ট call করার জন্য object তৈরি করতে হয় না। আবার ঐ class এর method/attribute লিখে যায়।

class এর name দিয়ে Direct access করা object লাগে না

Q. why method overloading is not possible by changing the return type of method only?

A. :

Compiletime Polimorphism:

Same name ও method ও class থেকে কোনটিকে call করবে তাহা decide হয় compile করার সময়ে এই কারণে compile time polimorphism.

Q. can we overload java main() method?

A yes, by method overloading. we can have any number of main methods in a class by method overloading. But JVM call main() method which receives String array as arguments only.

ex:

```
class TestOverloading4 {
    public static void main(String[] args) {
        System.out.println("Main with String[]");
    }
}
```

Super keyword in Java

we can call 3 types usages

(Super. color) parent calls এর attribute কে print করি

২য়টি child class এ attribute কে print করি কোনো

System.out.println (color)

২য়টি parent class কে call করতে চাই কোনো Super keyword use করতে হবে	২য়টি child class কে call করতে চাই কোনো আলাদাভাবে call করতে হবে

problem solve

```
class employee{
    int calculate_salary(){
        return 30000;
    }
}

class manager extends employee{
    int calculate_salary(){
        return 60000;
    }
}

class Developer extends Employee{
    int calculate_Salary(){
        return 50000;
    }
}

public main{
public static void main(String[] args){

    manager m = new manager();
    Developer D = new Developer();
```

```
System.out.println("Manager Salary: "+m.calculate_
                                    Salary());
System.out.println("Developer salary: "+D.calculate_salary
                                    ());
```

⇒

```
class product{
    String name;
    double price;

product (String name, double price){
        this.name = name;
        this.price = price;
    }

    double calculatePrice (){
        return price;
    }
}

class Electronics extends product{
    Electronics (String name, double price){
        super (name, price);
    }
```

```
@Override double calculatePrice (){
    return price - (price * 0.10);
    }
}

Class Clothing extends Product {
  Clothing (String name, double price){
   Super (name, price);
  }

  double calculateprice (){
    return price - (Price * 0.20);
  }
}

Public class Main{
   public static void main (String[] args){

Electronics e = new Electronics ("Laptop, 50000")
Clothing c = new Clothing ("Shirt, 2000);
System.out.println (Product : +e.name);
System.out.println ("original price:" + e.price);
```

```
System.out.println("After 10% discount: " + e.calculatePri.
                                                ());

System.out.println("product: " + c.name);

System.out.println("original price" + c.price)

System.out.println("after discount 20%:" + c
                    calculatePrice());
```

```
class Shape {
    String color;

    Shape (String color) {
        this.color = color;
    }

    Void display_color () {
        System.out.println ("color : " + color);
    }
}

class Rectangle extends Shape {
    double ~~weight~~ length;
    double width;

    Rectangle (String color, double length, double
                                    width) {
        Super (color);
        this.length = length;
        this.width = width;
    }
```

```
double calculate_Area (){
    return length * width ;
}
Void display (){
    display_color ();
    System.out.println ("Length : " + length);
    "    "    "    ("Width : " + width);
    "    "    "    ("Area : " + calculateArea);
}
}
Public class main {
    public static void main (String[] args) {
        Rectangle R = new Rectangle ("Blue, 10, 5);
        R.display ();
    }
}
```

Class hierarchy ⇒ Inheritance.

Garbage Collection

⇒ Obje The object becomes a candidate for automatic garbage collection.

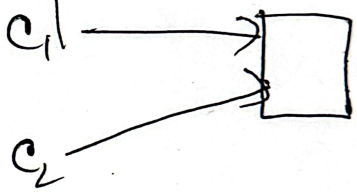
primitive Type : int i = i

```
circle c1 = new circle();
circle c2 = new circle()
```

c1 = null
↓
এখানে c1 কে garbage হিসাবে ধরে

```
c2 = c1
```

↳ c2 কে garbage হিসাবে ধরে



~~Class~~

এক প্যাকেজের ক্লাসকে অন্য প্যাকেজে import করতে হলে import করতে হবে।

```
import pack.*;
class B {
public static void main (String
```

Final কিওয়ার্ড দেওয়া কখনো update করা যাবে না।

```
final int a = 5;
final int b = 10;
```

Lab

default priva

Someclass

Q.1

Student
- name
- age
- CGPA

} Private

getter()

setter() → age ≥ 18 and ~~CGPA~~ age ≤ 30

→ CGPA ≥ 0.0 and CGPA ≤ 4.0

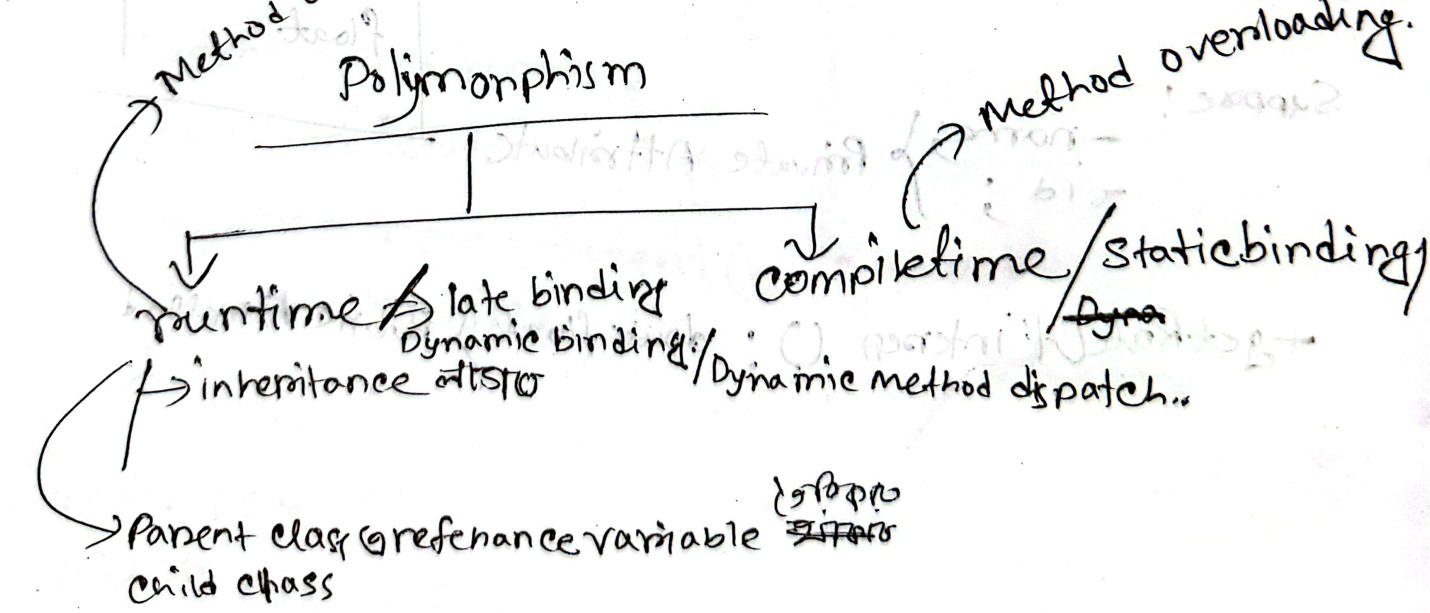
Encapsulation

long ⇒ integer

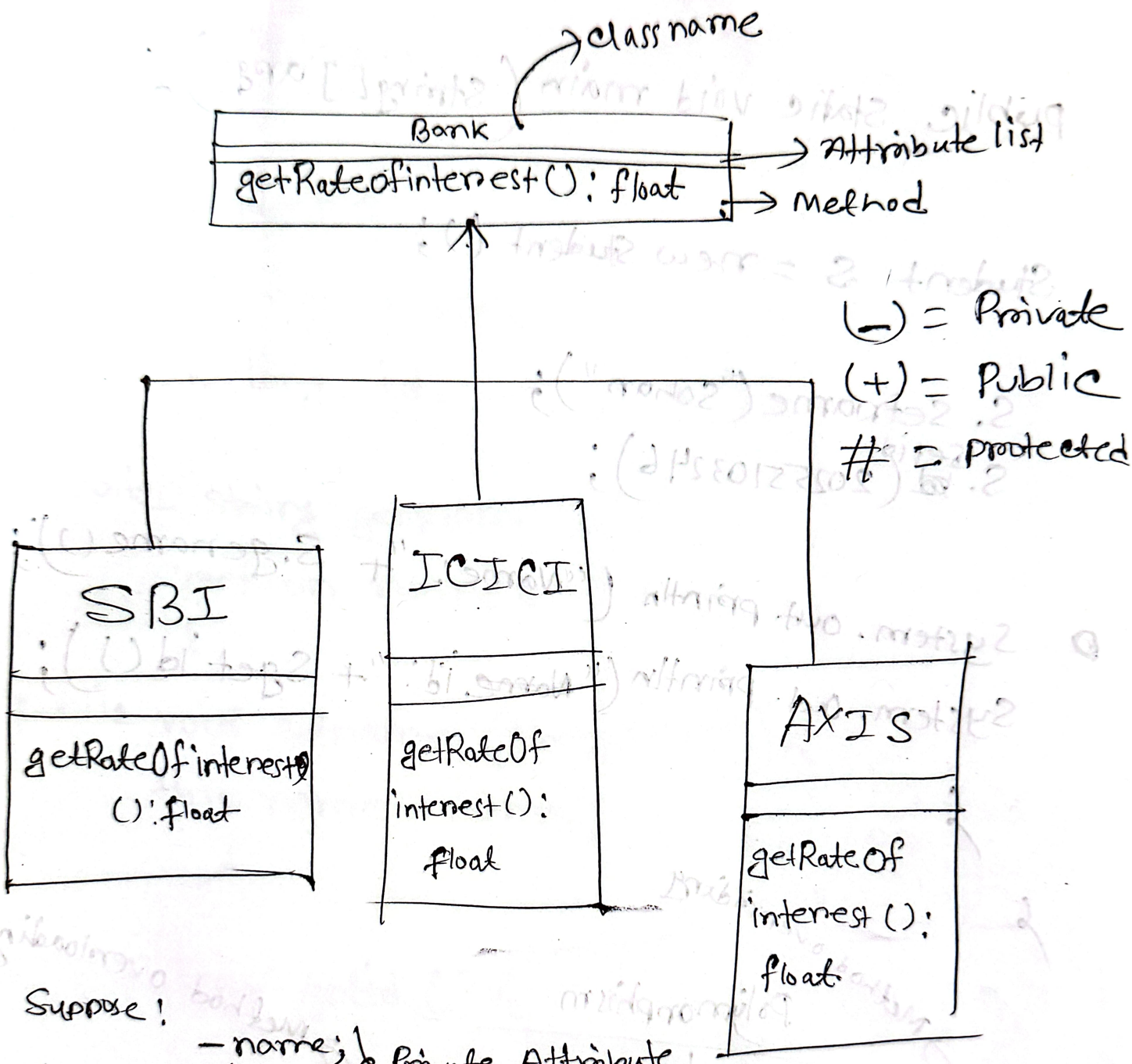
⇒

```
class student {
    private String name;
    private long id;

    public string getname() {
        return name;
    }
    public void setname (String name) {
        this.name = name;
    }

    public long getid() {
        return id;
    }
    public void setid (long id) {
        this.id = id;
    }
```

```
public class test {

    public static void main (String[] args) {

    Student s = new Student ();

      s. setname ("Sohan");
      s. setid (2025510324 6);

    System. out. println ("Name: " + s.getname());
    System. out. println ("Name id: " + sget id ());

    }
}
```



Class diagram



Suppose !

- name; } Private Attribute
- id ;

+getRateOf interest (): ~~doub~~ float } Public Method

1. Object passing and returning from Method.

⇒
```
class student{ int marks;
void function(student ob)
  { int m;
    m = this.marks + ob.marks
    System.out.println("The updated marks:"+ m);
  }
}

main (){

Student S1 = new student ();
student S2 = new student();
S1.marks = 10; S2.marks = 20;
   S1.function (S2);
```

(b)
```
class Student{ int marks;
Student function (Student ob) {
   Student S = new Student();
   return this;
  }
}
int main(){
 Student S1 = new Student()
 Stu  S2 = ' ' ;
S1.marks = 20 ; S2mark = 20
```

```
class rectangle {
    double width;
    double length;

rectangle (double width, double length) {
        this.width = width;
        this.length = length;
    }
double area () {
    return length*width ;
    }
void compareArea (rectangle r) {
    if (this.area() > r.area ()) {
    System.out.println ("Rectangle 1 has greater area.");
    }
  else if (this.area() < r.area ()) {
system.out.println ("Rectangle 2 has greater area.");
    }
else {
    system.out.println ("both has same area.");
    }
}
```

```
public class test{
public static void main (String[] args){

  rectangle r1 = new rectangle (10,5);
    rectangle r2 = new rectangle (20,5);

System.out.println("area of rectangle 1 : "+ r1.area());
System.out.println("area of rectangle 2 : "+ r2.area());

  r1.comparearea (r2);}
    }
}
```

```
class EBook{
    int pagesread;
    int totalpages;

    Static int totalEBooks = 0;

EBook (int pagesread, int totalpages){
    this.pagesread = pagesread;
    this.totalPages = totalpages;
        totalEBooks++;
    }

EBook incrementread(int p){
    pagesread = pagesread + p;
      return this;
    }

EBook decrementread(int p){
    pagesread = pagesread - p;
    return this;
    }

EBook remainingpages (){
    int remaining = totalpages - pagesread;
    System.out.println ("Total remaining page: "+remaining);
    return this;
    }
```

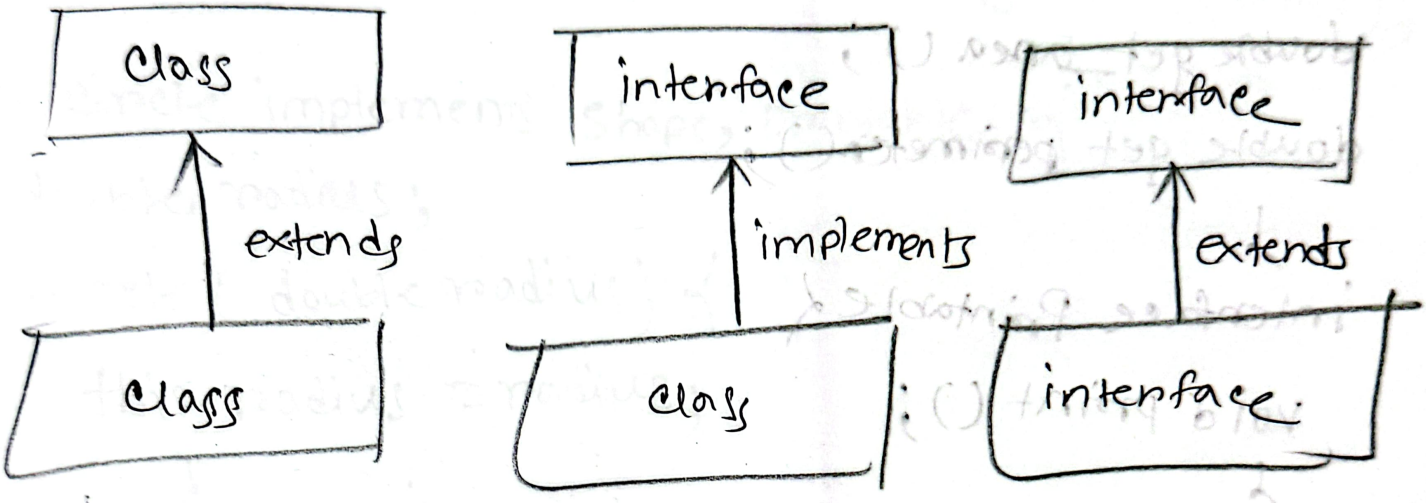
```
Static int getTotalEBooks(){
    return totalEBooks;
    }
}
public class main{
public static void main (String[] args){
    EBook e1 = new EBook(20, 300);
    EBook e2 = new EBook(50,350);

    e1. incrementread (10);
    e1. decrementread remainingpg();
    e1. decrementread (5);
    e1. remainingpg();
e2. increment (15);
e2. remainingpg();
e2. decrementread (7);
e2. remainingpg();
System.out.println ("Total EBook Objects: "+EBook.getTotalEBook());
    }
}
```

```
abstract class Shape {
    abstract void run();
}
class Honda Extends Bike {
void run() {system.out.println("running safely");}
}
public class main{
  public static void main (String args[]){
    Bike obj = new Honda();
    obj.run();
  }
}
```

Interface ⇒ we can use abstract method in it.
non abstract method is not allow.

⇒
```
interface Shape {
    void roun();
    void eat();
}
class child implements Shape {
        void run(){
        }
    void eat(){
    }
}
```

Relation Between classes and interfaces



⇒

```
interface Printable{
    void print();
}
class Printer implements Printable{
public void print(){ System.out.println.
```

চুরি Parent সাকাসাকাল আবার কাজ দিয়ে যথা 2টি
computer
Class implement printable, showable

```
interface shape{
double get_area();
double get_perimeter();
}
interface Printable{
  void print();
}
class Rectangle implements Shape, printable{
    double length;
    double width;
Rectangle (double length, double width){
    this.length = length;
    this.width = width;
}
public double get_area(){
    return length * width;
}
public double get_perimeter(){
  return 2*(length+width);
}
```

```
public void print() {
    System.out.println("Rectangle:");
    System.Out.Println("Area:" + get_Area());
    System.out.println("Perimeter:" + get_perimeter());

class circle implements Shape, printable {
    double radius;
    circle( double radius) {
        this.radius = radius;
    }
public (double get_area () { return

~~const~~
```

Constructor chaining

Person (name)

↑

Employee (ID, salary)

↑

Programmar (bonus)

Inner class

Class A { → inner class / (class এর ভিতর class / Nested class

Class B {...

}

}

এরপর inner class এর object create করতে হবে

Outerclass myouter = new outerclass();

Outerclass.Innerclass myInner = myouter.new Innerclass

System.out.println(myInner.y + myouter.x);

Array Object in Java

```
int[] arr = new int[5];   // array declaration
Student ob = new Student(10, "AAA");
```

Student class ला सुद्धा array declear करता येते

```
Student[] arr = new Student[5]
arr[0] = new Student(10, "AAA")

for (i=0; i<arr.length; i++) {
    arr[i];
}
```

```
Student[]
arr = new Student[5];

arr[0] = new Student(1, "aman");
arr[1] = new Student(2, "vaibhav");
arr[2] = new Student(3, "shikar");
arr[3] = new Student(4, "dharmesh");
arr[4] = new Student(5, "mohit");

for(i=0; i<arr.length; i++) {
    Sout("Element at "+i+": "+(arr[i].roll+" "+
        arr[i].name+")");
}
```

abstract class Shape

abstract get_area()
void display();

class Rec extends Shape
length, width
getArea(){

display()

lab

1.

```
abstract class Shape{

    abstract int get_Area()
    void display (){
     System. out. println ("shape area");
    }
}

Class rec extends Shape {
    int l,w;
  rec (int l, int w){
     this. l = l
     this. w = w.
    }
int get_Area (){
   return l* w
    }
Void display (){
    System.out.println ("length"+l)
    System . Out. Println (" width "+ w)
   System. out . println("Rectangle area: "+get_Area());
    }
}

Public class Mavenproject 8{
public static void main (String[] args){
Shape s=new rec (6,5);
 s.display.();
 }
}
```

Array দিয়ে করতে চাই অথবা আগের মূলের area
এক সময় যে করতে চাই same as pevious just
main function change

⇒

```
Public class Maven project {
    public static void main (String[] args){
        rec[] arr = new rec[3];
        arr[0] = new rec (2,3);
        arr[1] = new rec (9,3);
        arr[2] = new rec (5,6);
        for (int i=0; i<arr.length; i++){
System.out.println ("Rectangle"+i+ "the area:"
            + arr[i].get_area());
    }
}
```